

Các mô hình mạng tuyến tính (Linear Network Models)

Tối ưu hoá – AI203DV01
(Optimization)
Vũ Đình Khôi

Nội dung

- Giới thiệu
- Một số bài toán
 - Maximum Flow
 - Minimum Cost Flow
 - Transshipment
 - Shortest Paths

- Chapter 04. Serge Kruk. (2018). *Practical Python AI Projects: Mathematical Models of Optimization Problems with Google OR-Tools*. Apress
- <https://developers.google.com/optimization/flow>

**I find that I don't understand things unless I try
to program them.**



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- **Richard Feynman**

■ Mạng (network)

- Là một đối tượng bao gồm các nút (nodes) và các cung (arcs) mô tả mối quan hệ giữa các nút.
- Là công cụ toán học lâu đời giúp mô hình hóa và giải quyết các bài toán trong thực tế.

■ Các mô hình tối ưu dựa trên mạng lưới (network-based optimization models) thường có chung một đặc điểm thú vị (cùng với mô tả cấu trúc)

- *Nếu dữ liệu đầu vào (input data) là các số nguyên (integers) thì sẽ có một nghiệm tối ưu nguyên (integral optimal solution). Và các solver sẽ tìm ra nghiệm này.*
- Đây là một tính chất hữu ích (useful property) → cho phép mô hình hóa các biến/phần tử có thể đếm được (người, xe tải, gói dữ liệu) cũng như các biến có thể đo lường được (tiền, thời gian, nước)...

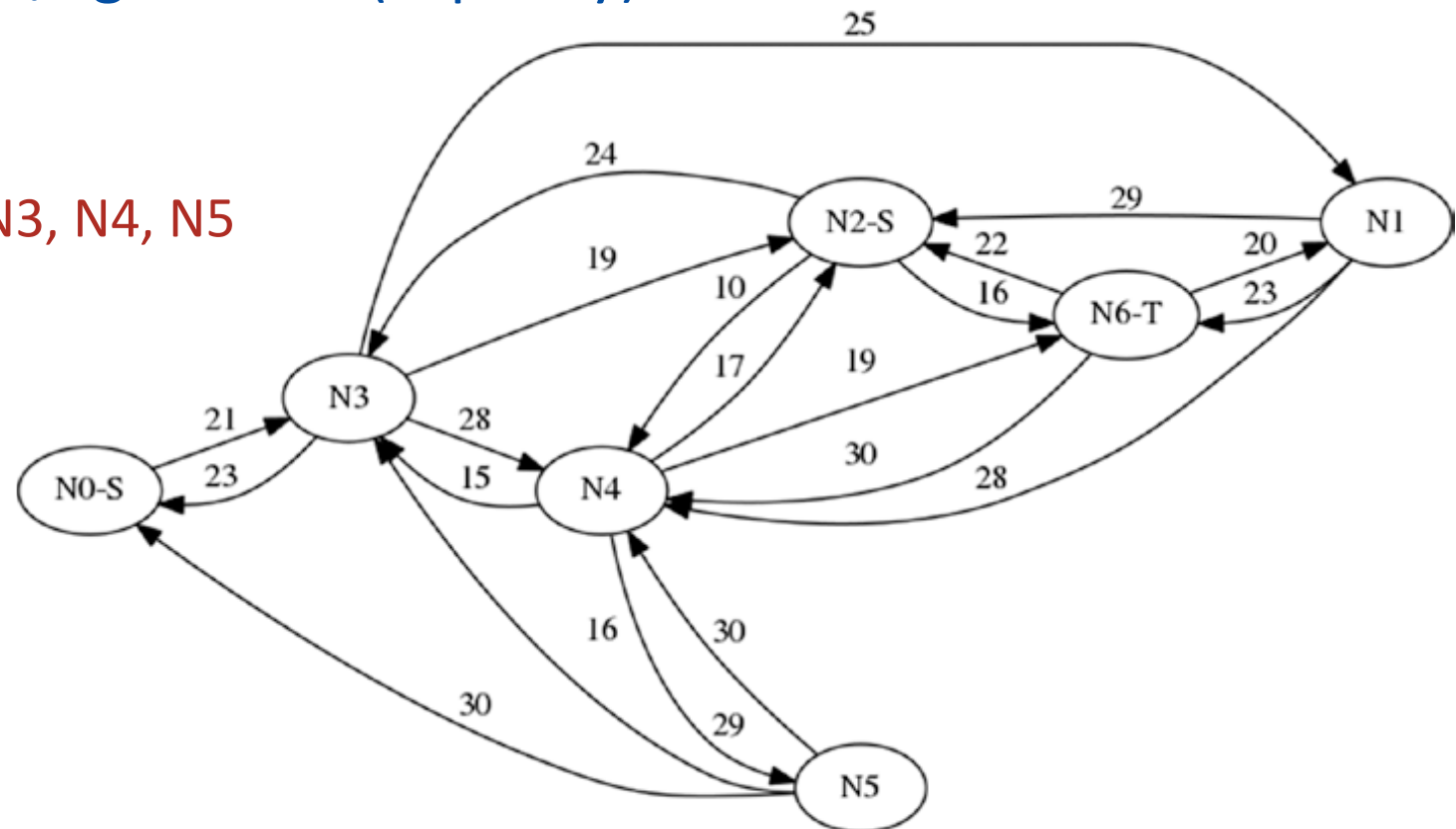
- Ví dụ bài toán phức tạp liên quan đến chuyển bay của tàu con thoi vũ trụ (space shuttle)
 - Các ràng buộc như khối lượng tàu, nhiên liệu, oxygen, các công việc thực hiện khi tàu hoạt động trên quỹ đạo (orbit)...
 - Có thể có hàng ngàn, thậm chí hàng trăm ngàn biến và ràng buộc
 - Câu hỏi mục tiêu (objective function): có thể mang theo bao nhiêu phi hành gia (astronauts) vào quỹ đạo?
 - Tất nhiên, NASA/SpaceX không thể chấp nhận nghiệm tối ưu là 2.5 phi hành gia 😊
 - → tầm quan trọng của tính nguyên (integrality) trong việc mô hình hóa.
- Điều quan trọng là chúng ta phải loại bỏ cách giải quyết hấp dẫn nhưng đầy sai lầm là làm tròn (rounding).
 - Hiếm khi việc làm tròn là có ích: nếu làm tròn nghiệm → nhiều ràng buộc có thể bị vi phạm → các quyết định sai lầm.

Maximum flow

- Bài toán tối đa lưu lượng (hay luồng cực đại)
- Các bài toán liên quan mạng lưới thường có một cấu trúc mà tính nguyên (integrality) của nghiệm luôn đảm bảo “miễn phí”.
 - Làm sao nhận diện các bài toán rơi vào dạng này → chỉ cần mô hình hóa các cấu trúc đặc biệt này.
- Dạng tổng quát của bài toán *network maximum flow*
 - Các luồng vật chất (substance flows) xuất phát từ các nguồn (sources) đi đến các đích (destinations) thông qua các kênh có trọng số khả tải/khả lưu (capacitated channels)
 - “substance flowing” không chỉ là vật chất như xăng, dầu, nước hay điện, có thể các gói dữ liệu (data packets) trên mạng máy tính hay video streams giữa các server với người dùng mạng.
 - Cố gắng tối đa lưu lượng/ tìm luồng chảy cực đại qua các kênh?

Maximum flow

- Các node được đánh dấu
 - nguồn (-S) hoặc đích (-T/Target). Đích còn gọi là thùng chứa (sinks)
- Mỗi cung (arc) là dung lượng khả lưu (capacity)
 - Sources: N0-S, N2-S
 - Targets (sinks): N6-T
 - Intermediate nodes: N1, N3, N4, N5



Maximum flow: Constructing a model

■ Decision variables

- Cách đơn giản và tự nhiên là sử dụng các biến 2 chiều (2-dimensional). Ví dụ có 2 source và 3 target → biểu diễn thành 6 biến ma trận 2×3 mô tả các luồng lưu lượng từ nguồn tới đích.
- Giá trị của các biến chính là lưu lượng (flow) trên các cung (arc)

$$x_{i,j} \quad \forall i \in N, \forall j \in N$$

- N: network
- Ví dụ $x_{1,3} = 25$ nghĩa là gửi 25 đơn vị (unit) lưu lượng từ node 1 đến node 3

■ Objective/objective function

- Tối đa lưu lượng (maximum flow) từ tất cả các nguồn đến các đích (sinks). Hay tổng lưu lượng ra khỏi các nguồn (flow out), hoặc tổng lưu lượng vào các đích (flow in).

$$\max \sum_{i \in S} \sum_{j \in N} x_{i,j} \quad \text{hoặc} \quad \max \sum_{j \in T} \sum_{i \in N} x_{i,j}$$

Maximum flow

- Vì bài toán có thể cho phép nhiều nguồn (multiple sources) và không cấm lưu lượng có thể từ nguồn này đến nguồn khác (dạng trung chuyển).
 - Do đó, phải cẩn thận khi tối đa “lưu lượng ròng” (net flow = out - in) ra khỏi các nguồn

$$\max \sum_{i \in S} \left(\sum_{j \in N} x_{i,j} - \sum_{j \in N} x_{j,i} \right)$$

- Tuy nhiên, cần lưu ý 2 vấn đề sau

- Các nguồn liên tiếp (chaining of sources): 2 units đến N2-T có thể là 2 units từ N1-S, hoặc 1 unit từ N0-S cộng với 1 unit từ N1-S.



- Các chu trình (cycles): nghiệm tối ưu có thể là 10 units đến N2-T là 10 unit từ N0-S thông qua node trung gian N1, hoặc 20 unit từ N0-S trừ đi 10 unit do N1 back lại.



Maximum flow

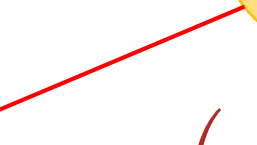
- Hai trường hợp trên minh họa bài toán có thể có nhiều nghiệm tối ưu (optimal solution), nhưng
 - giá trị tối ưu (optimal value) chỉ có một
 - Đặc biệt là nếu 2 solver khác nhau, hoặc 2 lần chạy khác nhau trên cùng một solver *có thể* trả về 2 lưu lượng khác nhau (different flows).
 - Làm sao để đảm bảo là solver trả về cùng lưu lượng nhất quán?
 - Trong tất cả các luồng tối ưu (optimal flow), chúng ta muốn có ít “flow” vào các nguồn nhất có thể.
- Mục tiêu kép (dual objective)
- Tối đa “net flow” ra (out) khỏi các nguồn
 - Tối thiểu “flow” vào (in) các nguồn

Maximum flow

- Sử dụng trick

$$\max f(x) \Leftrightarrow \min (-f(x))$$

- Mục tiêu max

$$\max \sum_{i \in S} \left(\sum_{j \in N} x_{i,j} - \sum_{j \in N} x_{j,i} \right)$$


- Mục tiêu min (chuyển sang max)

$$\min \sum_{i \in S} \sum_{j \in N} x_{j,i} = \max \left(- \sum_{i \in S} \sum_{j \in N} x_{j,i} \right)$$

- Ghép 2 mục tiêu max và min với nhau

$$\max \sum_{i \in S} \left(\sum_{j \in N} x_{i,j} - 2 \times \sum_{j \in N} x_{j,i} \right)$$

Maximum flow

■ Constraints

- Chỉ có một loại ràng buộc duy nhất là **bảo toàn lưu lượng** (conservation of flow).
- Đối với tất cả các node (trung gian), trừ node nguồn và đích (sink), lưu lượng vào (flow in) bằng lưu lượng ra (flow out)

$$\sum_{j \in N} x_{i,j} = \sum_{j \in N} x_{j,i} \quad \forall i \in N \setminus \{S \cup T\}$$

Maximum flow: Executable model

■ Input

- Mảng 2 chiều C (capacity) với các chỉ số (index) là node, giá trị là capacity giữa 2 node
- Mảng 1 chiều sources S
- Mảng 1 chiều targets T

■ Tham số unique

- unique = True: hàm mục tiêu tối đa “net flow” và tối thiểu flow vào các sources
- unique = False: chỉ đơn giản tối đa lưu lượng ra (flow out) của các sources

Maximum flow: Executable model

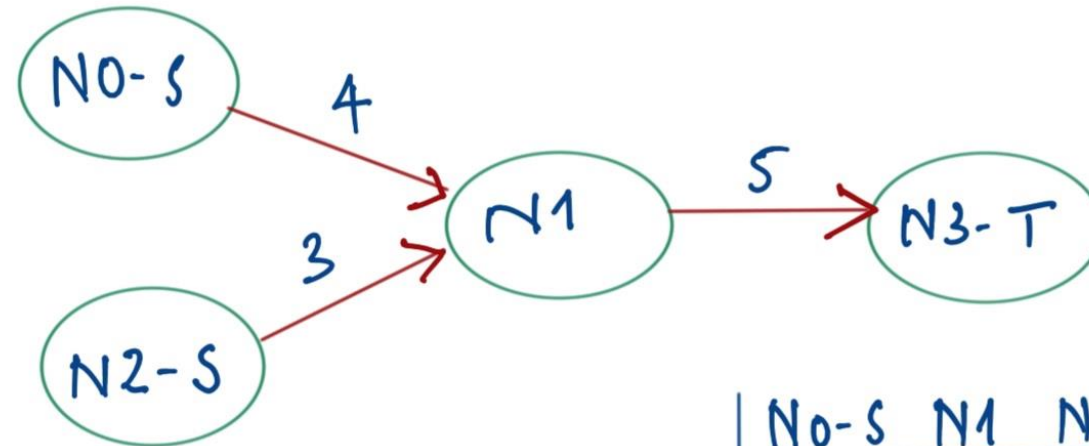
```
1 # maximum_flow.py: Bài toán cực đại luồng
2 from ortools.linear_solver import pywraplp
3 from my_or_tools import SolVal
4
5
6 # C: Capacity matrix, S: Sources, T: Targets/Sinks
7 def solve_maxflow(C, S, T, unique=True):
8     solver = pywraplp.Solver('Maximum Flow Problem',
9                             pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
10
11     n = len(C)
12     # Decision variables
13     # X[i][j]: lưu lượng (flow) thực sự đi từ node i đến node j
14     X = [[solver.NumVar(0, C[i][j], f'X_{i}_{j}')
15          for j in range(n)] for i in range(n)]
16     # B: tổng tất cả các dung lượng (capacity) có thể vận chuyển trong mạng
17     B = sum(C[i][j] for i in range(n) for j in range(n))
18     # cận trên (upper bound) tất nhiên không vượt quá B
19     flow_out = solver.NumVar(0, B, 'flow_out')
20     flow_in = solver.NumVar(0, B, 'flow_in')
```

Maximum flow: Executable model

```
21 # Constraints
22 for i in range(n):
23     if (i not in S) and (i not in T): # Intermediate node i
24         # Đối với các node trung gian, tổng lưu lượng ra phải bằng tổng lưu lượng vào
25         solver.Add(sum(X[i][j] for j in range(n)) == sum(X[j][i] for j in range(n)))
26
27 solver.Add(flow_out == sum(X[i][j] for i in S for j in range(n)))
28 solver.Add(flow_in == sum(X[j][i] for i in S for j in range(n)))
29
30 # Objective function
31 # net_flow: có thể có nhiều nghiệm tối ưu (optimal solution),
32 # nhưng giá trị tối ưu (optimal value) của hàm mục tiêu không đổi
33 net_flow = (flow_out - flow_in)
34 # dual_objective: đảm bảo nghiệm tối ưu là nhất quán (consistent)
35 dual_objective = (flow_out - 2 * flow_in)
36 solver.Maximize(dual_objective if unique else net_flow)
37 status = solver.Solve()
38
39 return status, SolVal(flow_out), SolVal(flow_in), SolVal(X)
40
```


Maximum flow: Executable model

■ Xét bài toán đơn giản



Capacity matrix :

	N0-S	N1	N2-S	N3-T
N0-S	0	4	0	0
N1	0	0	0	5
N2-S	0	3	0	0
N3-T	0	0	0	0

Maximum flow: Executable model

- Xét bài toán đơn giản

problem :
max flow từ các sources đến targets
Khi đó flow out của mỗi source là hằng số

Solution :
Rõ ràng max flow = 5
 $(N0-S, N1) = 2$; $(N2-S, N1) = 3$

Maximum flow: Executable model

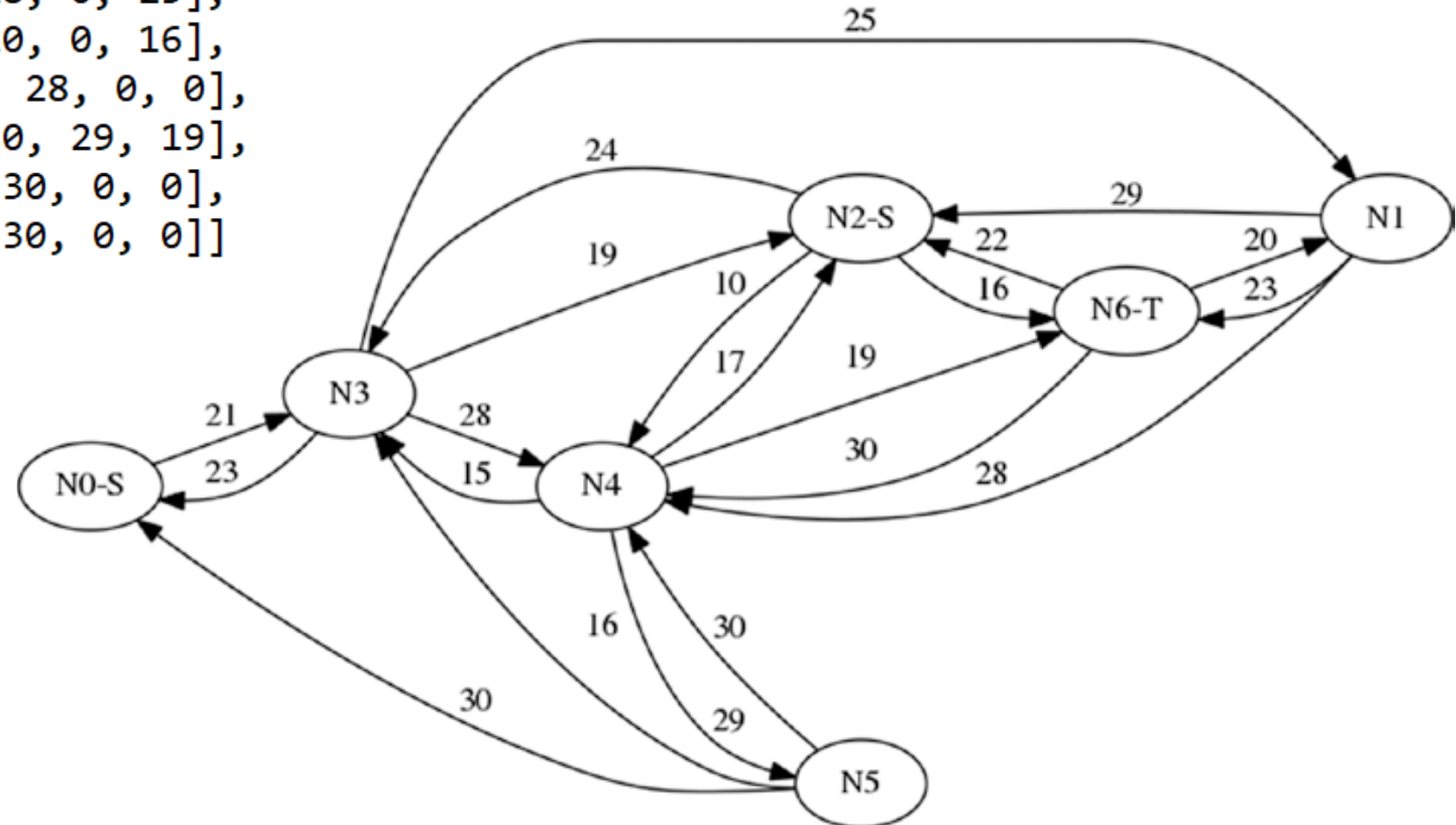
```
42 def main():
43     # Capacity matrix
44     C = [[0, 4, 0, 0],
45          [0, 0, 0, 5],
46          [0, 3, 0, 0],
47          [0, 0, 0, 0]]
48     # Sources and targets
49     S = [0, 2]
50     T = [3]
51
52     status, flow_out, flow_in, X = solve_maxflow(C, S, T, unique=False)
53     print('Value of objective function')
54     print('Flow out: {:.2f}'.format(flow_out))
55     print('Flow in: {:.2f}'.format(flow_in))
56     for i in range(len(X)):
57         print(X[i])
58
```

```
>>> %Run maximum_flow.py
Value of objective function
Flow out: 5.00
Flow in: 0.00
[0.0, 2.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 5.0]
[0.0, 3.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0]
```

Maximum flow: Executable model

Capacity matrix

```
C = [[0, 0, 0, 21, 0, 0, 0],
      [0, 0, 29, 0, 28, 0, 23],
      [0, 0, 0, 24, 10, 0, 16],
      [23, 25, 19, 0, 28, 0, 0],
      [0, 0, 17, 15, 0, 29, 19],
      [30, 0, 0, 16, 30, 0, 0],
      [0, 20, 22, 0, 30, 0, 0]]
```



Maximum flow: Executable model

```

42 def main():
43     # Capacity matrix
44     C = [[0, 0, 0, 21, 0, 0, 0],
45          [0, 0, 29, 0, 28, 0, 23],
46          [0, 0, 0, 24, 10, 0, 16],
47          [23, 25, 19, 0, 28, 0, 0],
48          [0, 0, 17, 15, 0, 29, 19],
49          [30, 0, 0, 16, 30, 0, 0],
50          [0, 20, 22, 0, 30, 0, 0]]
51     # Sources and targets
52     S = [0, 2]
53     T = [6]
54
55     status, flow_out, flow_in, X = solve_maxflow(C, S, T, unique=True)
56     print('Value of objective function')
57     print('Flow out: {:.2f}'.format(flow_out))
58     print('Flow in: {:.2f}'.format(flow_in))
59     for i in range(len(X)):
60         for j in range(len(X[i])):
61             print('{:2.0f}'.format(X[i][j]), end=' ')
62     print()

```

```
>>> %Run maximum_flow.py
```

Value of objective function

Flow out: 58.00

Flow in: 0.00

0	0	0	8	0	0	0
0	0	0	0	0	0	23
0	0	0	24	10	0	16
0	23	0	0	9	0	0
0	0	0	0	0	0	19
0	0	0	0	0	0	0
0	0	0	0	0	0	0

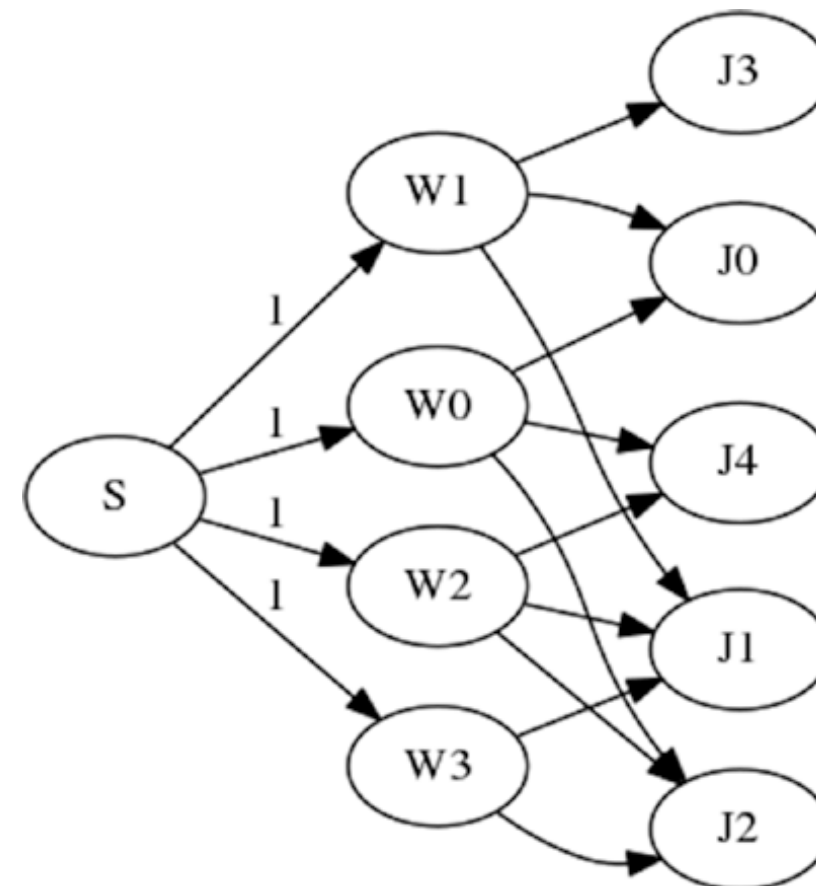
Maximum flow: Executable model

▪ Để ý

- Optimal solution: net flow gồm các biến `flow_out`, `flow_in` luôn là các số nguyên.

Maximum flow: Variations

- Mô hình các bài toán “Phân công việc” (assignment problems), có nhiều dạng khác nhau, ví dụ:
 - Với số lượng lao động/nhân công (workers) nhất định như con người, máy móc, nguồn lực cpu xử lý...
 - Số lượng nhất định các công việc (tasks/jobs) cần hoàn thành
- Model
 - Network gồm Một source node kết nối đến mỗi worker, các workers kết nối đến các jobs
 - Tối ưu việc phân công → nhiều công việc được hoàn thành nhất.



Maximum flow: Exercises

■ Đọc thêm

- <https://developers.google.com/optimization/flow/maxflow>

Minimum cost flow

- Bài toán luồng chi phí tối thiểu
 - Thuộc lớp bài toán nghiệm nguyên (class of problems where the integrality of the solution is guaranteed “for free”).
- Phát biểu bài toán/nguyên mẫu bài toán (prototypical example)
 - Các nhà máy điện (power plants) cung cấp điện cho nhiều thành phố
 - Mỗi nhà máy điện có công suất tối đa (maximum capacity) tính theo kW-h
 - Mỗi thành phố có nhu cầu điện cao điểm (peak demand), tất cả các thành phố có cùng giờ cao điểm.
 - Chi phí truyền tải điện từ nhà máy đến các thành phố là khác nhau (dollars/kW-h), phụ thuộc vào chi phí sản xuất, bảo trì nhà máy, đường dây, hạ tầng truyền tải, khoảng cách...

Minimum cost flow

■ Câu hỏi

- Lượng điện truyền tải từ mỗi nhà máy đến mỗi thành phố là bao nhiêu để đáp ứng nhu cầu cao điểm với chi phí thấp nhất (minimum cost)?

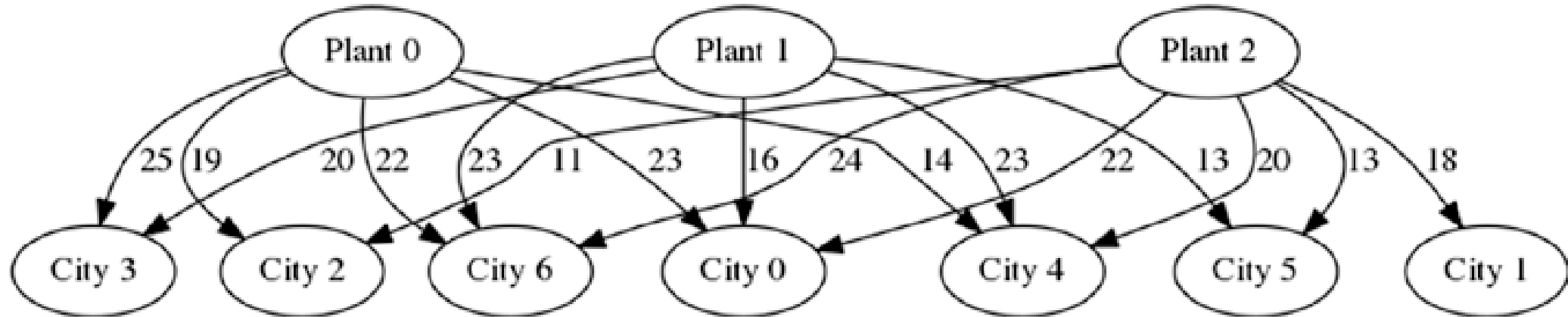
Table 4-3. Example of Electrical Distribution Cost

From/To	City 0	City 1	City 2	City 3	City 4	City 5	City 6	Supply
Plant 0	23		19	25	14		22	551
Plant 1	16			20	23	13	23	689
Plant 2	22	18	11		20	13	24	634
Demand	288	234	236	231	247	262	281	

Minimum cost flow: Constructing a model

■ Network

- Tạo đồ thị ghép đôi (bipartite graph) từ bảng chi phí.



■ Decision variables

- Các biến 2 chiều mô tả số lượng kW-h truyền từ nhà máy đến thành phố

$$x_{i,j} \quad \forall i \in P, \forall j \in C$$

- P: tập hợp các nhà máy phát điện (Plants)
- C: tập hợp các thành phố (Cities)

Minimum cost flow: Constructing a model

■ Objective

- Tối thiểu chi phí
- Hàm mục tiêu (objective function)

$$\min \sum_{i \in P} \sum_{j \in C} C_{i,j} \times x_{i,j}$$

- $C_{i,j}$: chi phí 1 kW-h từ nhà máy i đến thành phố j

■ Constraints

- Công suất cung tối đa (maximum capacity) của nhà máy

$$\sum_j x_{i,j} \leq S_i \quad \forall i \in P$$

- Nhu cầu cao điểm (peak demand) của từng thành phố

$$\sum_i x_{i,j} = D_j \quad \forall j \in C$$

Minimum cost flow: Executable model

```
1 # Power Supply Model (Minimum Cost Flow Problem)
2 # (minimum_cost.py)
3
4 from ortools.linear_solver import pywraplp
5 from my_or_tools import SolVal, ObjVal
6
7
8 # D: bảng/ma trận (2 chiều )chi phí
9 def solve_mincost(D):
10     solver = pywraplp.Solver('Minimum Cost Problem',
11                             pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
12     m = len(D) - 1 # số lượng nhà máy, trừ dòng Demand cuối cùng
13     n = len(D[0]) - 1 # số lượng thành phố, trừ cột Supply cuối cùng
14
15     B = sum(D[-1][j] for j in range(n)) # tổng cầu cao điểm (peak demand)
16     # Decision variables
17     # X[i][j]: lưu lượng (flow) thực sự đi từ nhà máy i đến thành phố j
18     X = [[solver.NumVar(0, B if D[i][j] else 0, f'X_{i}_{j}')]
19           for j in range(n)] for i in range(m)]
20
```

Minimum cost flow: Executable model

```

21 # Constraints
22 # Tổng cung của từng nhà máy không vượt quá maximum supply/capacity
23 for i in range(m):
24     solver.Add(sum(X[i][j] for j in range(n)) <= D[i][-1])
25 # Tổng cầu của từng thành phố giờ cao điểm (peak demand)
26 for j in range(n):
27     solver.Add(sum(X[i][j] for i in range(m)) == D[-1][j])
28 # Tổng chi phí
29 Cost = solver.Sum(X[i][j] * D[i][j] for i in range(m) for j in range(n))
30 solver.Minimize(Cost)
31 status = solver.Solve()
32
33 return status, ObjVal(solver), SolVal(X)
34

```

Điều gì xảy
ra nếu ràng
buộc này là
dấu ==

Minimum cost flow: Executable model

```
35
36 def main():
37     # Cost matrix
38     D = [[23, 0, 19, 25, 14, 0, 281, 551],
39          [16, 0, 0, 20, 23, 13, 0, 689],
40          [22, 18, 11, 0, 20, 13, 0, 634],
41          [288, 234, 236, 231, 247, 262, 281, 0]]
42
43     status, min_cost, X = solve_mincost(D)
44     print('Minimum cost: {:.2f}'.format(min_cost))
45     for i in range(len(X)):
46         print(X[i])
47
```

```
>>> %Run minimum_cost.py
Minimum cost: 101861.00
[0.0, 0.0, 0.0, 0.0, 247.00000000000003, 0.0, 281.0]
[288.0, 0.0, 0.0, 231.0, 0.0, 170.00000000000003, 0.0]
[0.0, 234.0, 236.0, 0.0, 0.0, 91.99999999999997, 0.0]
```

Minimum cost flow: Executable model

Table 4-4. *Optimal Solution to the Power Distribution Problem*

From/To	City 0	City 1	City 2	City 3	City 4	City 5	City 6	Total
Plant 0					247		281	528
Plant 1	288			231		170		689
Plant 2		234	236			92		562
Total	288	234	236	231	247	262	281	

Minimum cost flow: Variations

- Biến thể đơn giản nhất của bài toán là thêm các “capacities” vào các cung (arcs)
 - Ví dụ, lượng truyền tải điện không được vượt quá khả năng truyền tải của hạ tầng
- Biến thể thú vị khác là dàn trải (spreading) các nguồn (sources)
 - Ví dụ, để giảm thiểu rủi ro (an toàn) khi truyền tải, mỗi thành phố chỉ nhận tối đa 60% nhu cầu điện năng (peak demand) từ một nguồn/nhà máy → nghĩa là nguồn điện cung cấp cho nó tối thiểu phải từ 2 nhà máy trở lên.

$$x_{i,j} \leq A_{i,j} \quad \forall i \in P, \forall j \in C$$

- A: capacity matrix

$$x_{i,j} \leq 0.6 \times D_j \quad \forall i \in P, \forall j \in C$$

Minimum cost flow: Exercises

- Thay vì xử lý trên các luồng (flow) có thể chuyển thành bài toán phân công (assignment)
 - Cho tập cố định nhân công (workers) và lương giờ với tập các công việc (jobs) → phân công worker nào cho job nào để tối thiểu chi phí (cost)
- Ví dụ
 - Bảng chi phí di chuyển (travel cost) của các nhóm (ở các thành phố khác nhau) với các khách hàng (ở những địa điểm khác nhau) → tối thiểu chi phí di chuyển?

	Customer 0	Customer 1	Customer 2	Supply
Team 0	25	30	20	1
Team 1	20	15	35	1
Team 2	18	19	28	1
Demand	1	1	1	

Minimum cost flow: Exercises

■ Đọc thêm

- <https://developers.google.com/optimization/flow/mincostflow>
- https://developers.google.com/optimization/flow/assignment_min_cost_flow

■ Bài toán chuyển tàu

- Tập các nodes cùng với chi phí vận chuyển
- Tập con (subset) các nodes là nhà cung cấp (Suppliers)
- Tập con (subset) các nodes là khách hàng (Consumers)
- Các nodes còn lại dùng để vận chuyển nguyên liệu (materials) nhưng không sản xuất hay tiêu thụ
- Vậy có 3 loại nodes: producers/suppliers, consumers, and intermediate
- Trong đó supplier tương tự như source, consumer như sink (target)
- Một node có thể nhận nguyên liệu → thêm vào đó số lượng của mình → chuyển (delivery) kết quả cho node khác (là consumer node hoặc transshipment node).
- Và tổng supply = tổng demand hoặc bài toán không khả thi (infeasible)

→ tối ưu chi phí vận chuyển từ các node Suppliers đến các node Consumers

Transshipment

Table 4-6. *Example of Transshipment Distribution Cost Over a Network*

From/To	N0	N1	N2	N3	N4	N5	N6	N7	Supply
N0					17	10	19		
N1	23			28		23			
N2	29				30	25	25		680
N3					17	15	19	29	
N4		16							
N5	22				25			18	540
N6	25	29	16			22			
N7			30		10		27		
Demand	241			164	239		152	424	

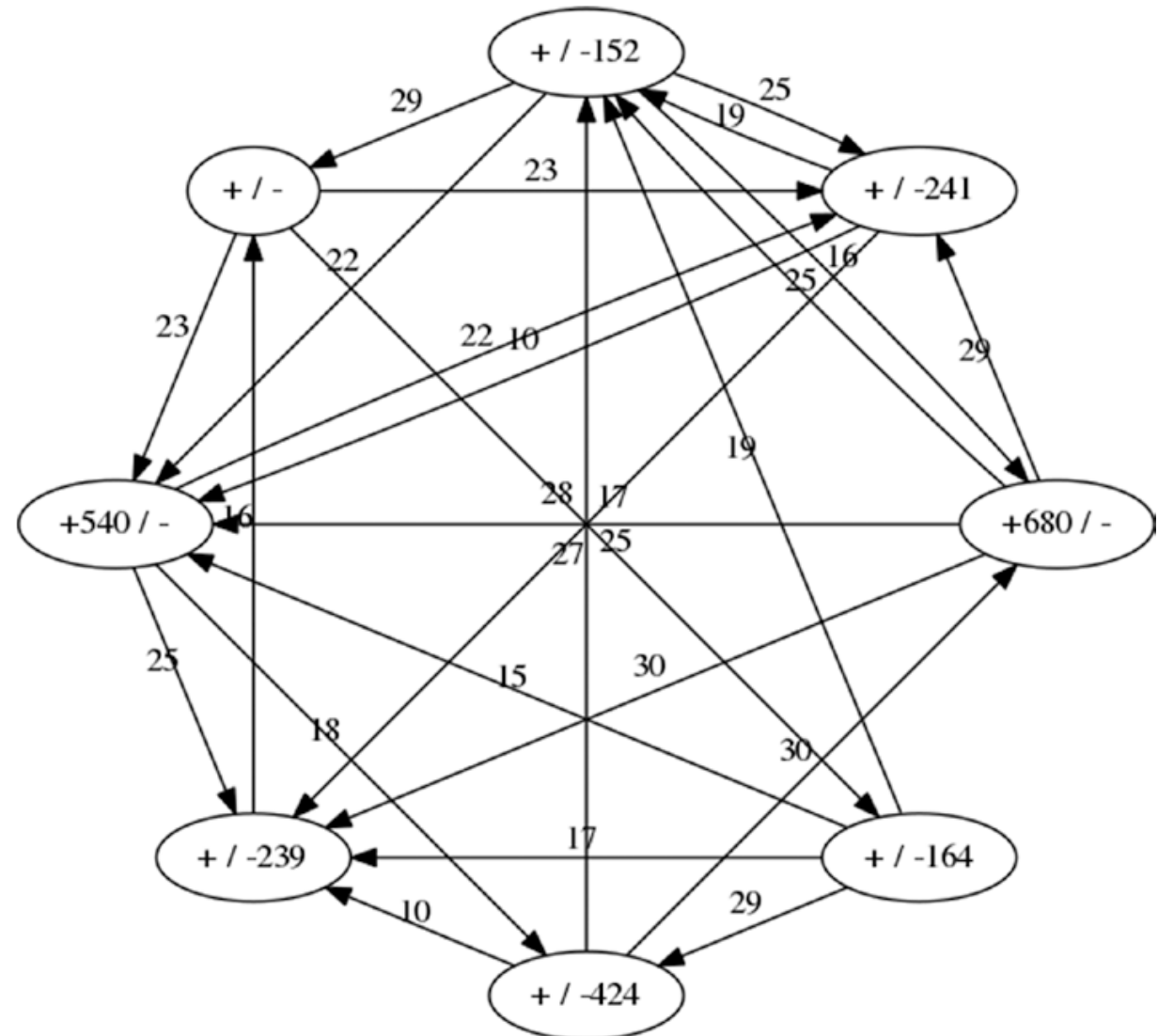
Sản lượng một node có thể sản xuất/cung cấp

Nhu cầu nguyên liệu tối đa của mỗi node

Transshipment: Constructing a model

■ Có thể trình bày dưới dạng đồ thị trực quan (hình bên)

- Mũi tên: chi phí vận chuyển nguyên liệu
- +: supply node
- -: demand node



Transshipment: Constructing a model

■ Decision variables

- Khối lượng/số lượng hàng cần vận chuyển (ship/delivery) từ node i đến node j

$$x_{i,j} \quad \forall i \in N, \forall j \in N$$

■ Objective

- Tối thiểu chi phí (dollars)

$$\min \sum_i \sum_j C_{i,j} \times x_{i,j}$$

Transshipment: Constructing a model

■ Constraints

- Có thể tạo ràng buộc riêng cho từng loại node (producers, consumers hay intermediate) như bài toán minimum cost flow.
- Tuy nhiên có thể tổng quát như sau cho tất cả loại node: tổng lượng ra – tổng lượng vào = cung (supply) – cầu (demand)

$$\sum_j x_{i,j} - \sum_j x_{j,i} = S_i - D_i \quad \forall i \in N$$

- Lưu ý, các producers thì $D = 0$ và $S \neq 0$, các consumers thì ngược lại $D \neq 0$, $S = 0$. Các intermediate thì cả $S - D = 0$
- Ràng buộc trên về tổng quát được gọi là ràng buộc “bảo toàn dòng chảy” (conservation of flow).

Transshipment: Executable model

```
1 """Bài toán sang tàu - Transshipment Problem (transshipment.py)
2 thuộc lớp Network Flow Problem"""
3 from ortools.linear_solver import pywraplp
4 from my_or_tools import SolVal, ObjVal
5 # D: cost matrix, với hàng cuối cùng là demand, cột cuối cùng là supply
6 def solve_transshipment(D):
7     solver = pywraplp.Solver('Transshipment Problem',
8                               pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
9
10    n = len(D) - 1 #
11    B = sum(D[-1][j] for j in range(n)) # tổng demand
12
13    # Decision variables
14    G = [[solver.NumVar(0, B if D[i][j] else 0, f'G_{i}_{j}')
15          for j in range(n)] for i in range(n)]
16
17    # Constraints
18    # Đối với mỗi loại node: (tổng ra - tổng vào) = (supply - demand)
19    for i in range(n):
20        solver.Add(sum(G[i][j] for j in range(n)) - sum(G[j][i] for j in range(n))
21                    == D[i][-1] - D[-1][i])
```

Transshipment: Executable model

```
22     # Objective function
23     Cost = solver.Sum(G[i][j] * D[i][j] for i in range(n) for j in range(n))
24
25     solver.Minimize(Cost)
26     status = solver.Solve()
27
28     return status, ObjVal(solver), SolVal(G)
29
30
31 def main():
32     # Cost matrix
33     D = [[0, 0, 0, 0, 17, 10, 19, 0, 0],
34          [23, 0, 0, 0, 28, 0, 23, 0, 0],
35          [29, 0, 0, 0, 30, 25, 25, 0, 680],
36          [0, 0, 0, 0, 17, 15, 19, 29, 0],
37          [0, 16, 0, 0, 0, 0, 0, 0, 0],
38          [22, 0, 0, 0, 25, 0, 0, 18, 540],
39          [25, 29, 16, 0, 0, 22, 0, 0, 0],
40          [0, 0, 30, 0, 10, 0, 27, 0, 0],
41          [241, 0, 0, 164, 239, 0, 152, 424, 0]]
```

Transshipment: Executable model

```
43 status, min_cost, G = solve_transshipment(D)
44 print('Minimum cost: {:.2f}'.format(min_cost))
45 for i in range(len(G)):
46     print(G[i])
47
```

```
>>> %Run transshipment.py
Minimum cost: 36915.00
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 164.0, 0.0, 0.0, 0.0, 0.0]
[124.99999999999994, 0.0, 0.0, 0.0, 403.0, 0.0, 152.00000000000001, 0.0]
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 164.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[116.00000000000006, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 423.99999999999994]
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

Transshipment: Executable model

■ Kết quả?

- N1: intermediate nên $Out = In$
- N4: consumer nên $In = 403$ nhưng $Out = 164$ --> chênh lệch Demand = $403 - 164 = 239$

Table 4-7. Optimal Solution to the Transshipment Problem

From/To	N0	N1	N2	N3	N4	N5	N6	N7	Out
N0									
N1				164					164
N2	125				403		152		680
N3									
N4		164							164
N5	116							424	540
N6									
N7									
In	241	164		164	403		152	424	

Transshipment: Variations

■ Một trong các biến thể của bài toán có thể giải (possible variation)

- Khi supply và demand không cân bằng (unbalanced)
- Các suppliers/producing nodes có thêm số lượng sản xuất tối đa (maximum production capacity), chỉ có phần demand là thỏa điều kiện.
- Khi này các ràng buộc phải tách riêng cho từng loại nodes

$$\sum_j x_{i,j} - \sum_j x_{j,i} \leq S_i \quad \forall \{i \in N \mid S_i > 0\}$$

■ Biến thể khác

- Mỗi cung (arc) có thêm số lượng capacity, giới hạn số lượng đi qua chúng.

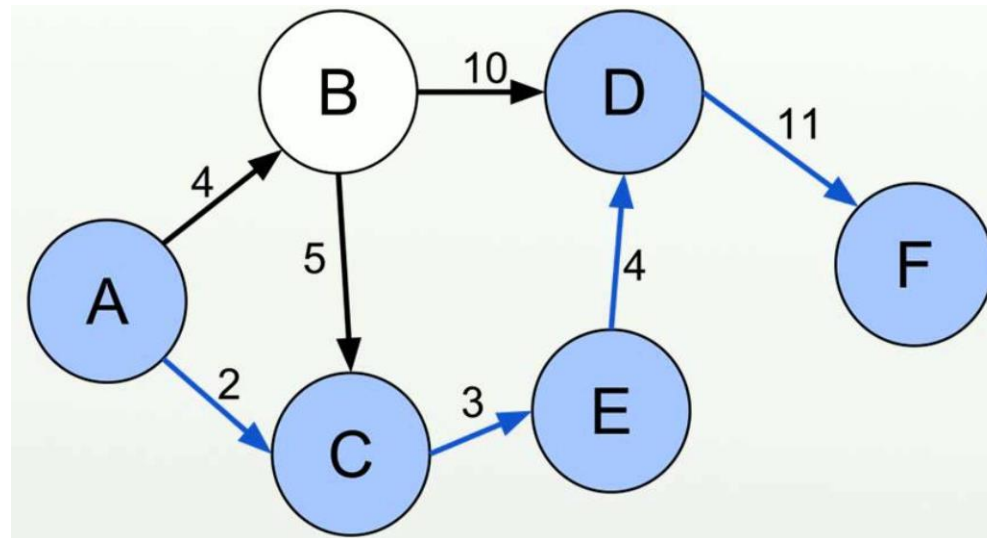
$$x_{i,j} \leq C_{i,j}$$

- C: capacity matrix

Shortest paths

■ Bài toán tìm đường đi ngắn nhất

- Có thể theo khoảng cách (distance) hoặc thời gian (time)
- Đồ thị (graph) với các node là các điểm với khoảng cách là trọng số (weight/capacity) của các cung (arcs).



- Điều thú vị là bài toán này có thể được mô hình và giải rất hiệu quả như bài toán network flow.

Shortest paths

- Ma trận khoảng cách (distance matrix)
 - Biểu diễn khoảng cách giữa các điểm/đỉnh
- Tìm dãy liên tiếp (sequence) các đỉnh giữa điểm bắt đầu và điểm kết thúc
 - → cực tiểu tổng khoảng cách giữa các đỉnh này?
- Tìm cách xây dựng mô hình (model) và các ràng buộc (constraints)

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P0		46	17	24	51								
P1	46				31	33		54					
P2		38			34	31			51				
P3	24				33			17	49	31			
P4	51					4			18	39	60		
P5	48				4		4	27		35	57	51	
P6				33	1						59		
P7		54	26		32	27	31			14	42	66	
P8			51	49	18	20	17	43			57	32	
P9					39	35		14			28		
P10					60							58	6
P11									32	61	58		56
P12									59			56	

Shortest paths: Constructing a model

- **P** là tập các điểm cho trước
 - Nghiệm của bài toán là tập con (subset) của B đi từ điểm đầu đến điểm cuối.
- **Vậy các biến quyết định (decision variables) của mô hình là gì?**
 - Đây là trick: mỗi cung trên đồ thị sẽ có một biến quyết định xem cung đó có thuộc về đường đi ngắn nhất hay không?
 - Do đó, biến này có giá trị bool: nếu cung thuộc đường đi ngắn nhất → biến nhận giá trị 1, ngược lại là 0
 - Ma trận chứa các biến quyết định tương tự như ma trận khoảng cách nhưng các cung thuộc được đi ngắn nhất (nghiệm) có trọng số là 1
- **Decision variables**

$$x_{i,j} \in [0, 1] \quad \forall i \in P, \forall j \in P$$

Shortest paths: Constructing a model

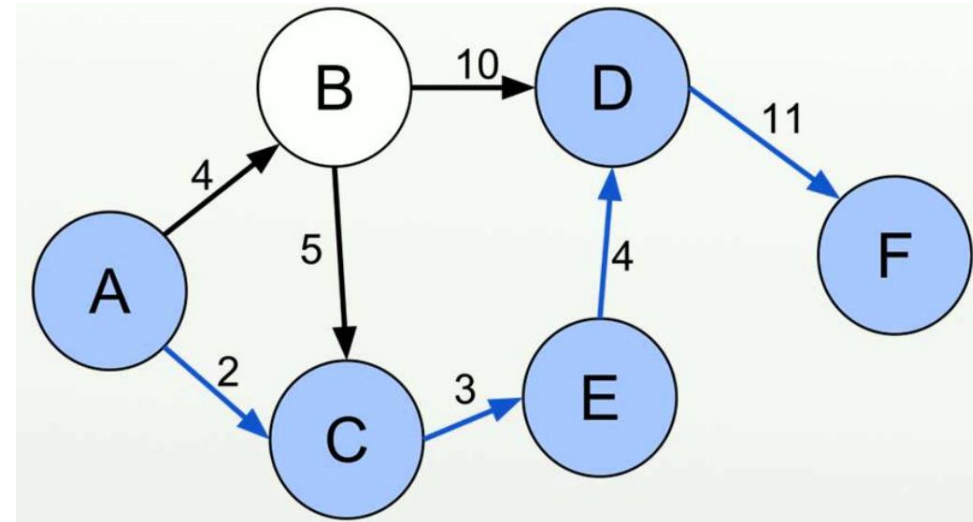
■ Các đường đi có thể

- $A \rightarrow B \rightarrow D \rightarrow F$: 25
- $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$: 27
- $A \rightarrow C \rightarrow E \rightarrow D \rightarrow F$: 20

■ Nghiệm

- Trên ma trận biến quyết định

Decision Variables	A	B	C	D	E	F
A	0	0	1	0	0	0
B	0	0	0	0	0	0
C	0	0	0	0	1	0
D	0	0	0	0	0	1
E	0	0	0	1	0	0
F	0	0	0	0	0	0



Points	A	B	C	D	E	F
A		4	2			
B			5	10		
C					3	
D						11
E				4		
F						

Shortest paths: Constructing a model

- Lưu ý khi khởi tạo các biến quyết định ban đầu
 - Các đỉnh không kết nối (không có đường đi) $\rightarrow x_{i,j} = 0$
 - Các đỉnh có kết nối (có trọng số khoảng cách) $\rightarrow x_{i,j} = 1$

Decision Variables	A	B	C	D	E	F
A	0	1	1	0	0	0
B	0	0	1	1	0	0
C	0	0	0	0	1	0
D	0	0	0	0	0	1
E	0	0	0	1	0	0
F	0	0	0	0	0	0

Sau khi tối ưu



Decision Variables	A	B	C	D	E	F
A	0	0	1	0	0	0
B	0	0	0	0	0	0
C	0	0	0	0	1	0
D	0	0	0	0	0	1
E	0	0	0	1	0	0
F	0	0	0	0	0	0

Shortest paths: Constructing a model

■ Objective

- Chúng ta xem như đây là một luồng đơn vị (unit flow) xuyên qua đồ thị, bắt đầu từ điểm đầu (như là source của giá trị 1) và điểm cuối (như là sink của giá trị 1).

$$\min \sum_i \sum_j D_{i,j} \times x_{i,j}$$

■ Constraints

- Điểm đầu (như source trong network flow) có thể có nhiều đường (cung) đi ra, nhưng chỉ có 1 cung thuộc đường đi ngắn nhất. Tất nhiên, source không có input flow.

$$\sum_j x_{i,j} = 1 \quad i \in [start\ node], \forall j \in P \setminus i$$
$$\sum_i x_{j,i} = 0 \quad i \in [start\ node], \forall j \in P \setminus i$$

Shortest paths: Constructing a model

■ Constraints

- Điểm cuối (như sink trong network flow) có thể có nhiều đường (cung) đi vào, nhưng chỉ có 1 cung thuộc đường đi ngắn nhất. Tất nhiên, sink không có output flow.

$$\sum_j x_{j,i} = 1 \quad i \in [end\ node], \forall j \in P \setminus i$$

$$\sum_i x_{i,j} = 0 \quad i \in [end\ node], \forall j \in P \setminus i$$

- Các điểm trung gian theo định luật bảo toàn lưu lượng (conservation of flow) có input flow và output flow

$$\sum_{j \in P} x_{i,j} = \sum_{j \in P} x_{j,i} \quad \forall i \in P \setminus \{start, end\}$$

Shortest paths: Executable model

```
1  """
2  Bài toán tìm đường đi ngắn nhất giữa 2 đỉnh trong đồ thị
3  Sử dụng tối ưu hóa như bài toán Network Flow
4  (shortest_path.py)
5  """
6  from ortools.linear_solver import pywraplp
7  from my_or_tools import *
8
9
10 # D: distance matrix
11 def solve_shortest_path(D, start=None, end=None):
12     solver = pywraplp.Solver('Shortest Path Problem',
13                               pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
14
15     n = len(D)
16     if start is None:
17         start = 0
18     if end is None:
19         end = n - 1
```

Shortest paths: Executable model



```
20 # Decision variables
21 # X[i][j] thuộc [0, 1] nếu có cạnh từ i đến j
22 # ngược lại X[i][j] thuộc [0, 0] nghĩa là = 0
23 X = [[solver.NumVar(0, 0 if D[i][j] is None else 1, '')
24       for j in range(n)] for i in range(n)]
25
26 # Constraints
27 for i in range(n):
28     # nếu i là điểm đầu (như source) có thể có nhiều đường đi ra
29     # nhưng chỉ có 1 đường thuộc đường đi ngắn nhất (sum = 1)
30     # và source không có đường đi vào (sum = 0)
31     if i == start:
32         solver.Add(sum(X[start][j] for j in range(n)) == 1)
33         solver.Add(sum(X[j][start] for j in range(n)) == 0)
34     # sink: chiếm 1 đường đi vào và không có đường đi ra
35     elif i == end:
36         solver.Add(sum(X[j][end] for j in range(n)) == 1)
37         solver.Add(sum(X[end][j] for j in range(n)) == 0)
38     else: # các điểm trung gian theo định luật bảo toàn luồng
39         solver.Add(sum(X[i][j] for j in range(n)) ==
40                     sum(X[j][i] for j in range(n)))
```

Shortest paths: Executable model

```
41
42 # Objective function
43 solver.Minimize(solver.Sum(X[i][j] * (0 if D[i][j] is None else D[i][j])
44                  for i in range(n) for j in range(n)))
45 status = solver.Solve()
46
47 # Trình bày kết quả
48 path, cost, cumulative_cost, node = [start], [0], [0], start
49 while status == 0 and node != end and len(path) < n:
50     next_node = [i for i in range(n) if X[node][i].solution_value() == 1][0]
51     path.append(next_node)
52     cost.append(D[node][next_node])
53     cumulative_cost.append(cumulative_cost[-1] + cost[-1])
54     node = next_node
55
56 return status, ObjVal(solver), SolVal(X), path, cost, cumulative_cost
57
```

Shortest paths: Executable model

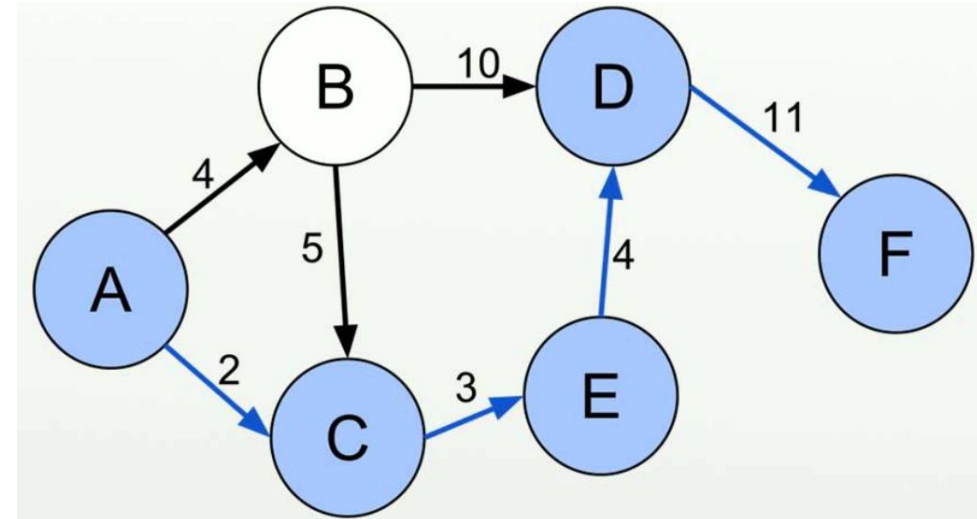
```
59 def main():
60     # Distance matrix
61     D = [[None, 4, 2, None, None, None],
62          [None, None, 5, 10, None, None],
63          [None, None, None, None, 3, None],
64          [None, None, None, None, None, 11],
65          [None, None, None, 4, None, None],
66          [None, None, None, None, None, None]]
67
68     status, obj_val, X, path, cost, cumulative_cost = solve_shortest_path(D)
69     if status == 0:
70         print('Path:', path)
71         print('Cost:', cost)
72         print('Cumulative cost:', cumulative_cost)
73         print('Objective value:', obj_val)
74         print('Decision variables (X):')
75         for i in range(len(X)):
76             print(X[i])
77     else:
78         print('No solution')
```


Shortest paths: Executable model

```
>>> %Run shortest_path.py
```

```
Path: [0, 2, 4, 3, 5]
Cost: [0, 2, 3, 4, 11]
Cumulative cost: [0, 2, 5, 9, 20]
Objective value: 20.0
Decision variables (X):
[0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 0]
```

Ký hiệu các node tương ứng:
0: A, 1: B, 2: C, 3: D, 4: E, 5: F



Shortest paths: Exercises

■ Viết chương trình

- Đọc distance matrix từ file csv
- Tìm đường đi ngắn
- Hiển thị kết quả

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P0		46	17	24	51								
P1	46				31	33		54					
P2		38			34	31			51				
P3	24				33			17	49	31			
P4	51					4			18	39	60		
P5	48				4		4	27		35	57	51	
P6				33	1						59		
P7		54	26		32	27	31			14	42	66	
P8			51	49	18	20	17	43			57	32	
P9					39	35		14			28		
P10					60							58	6
P11									32	61	58		56
P12									59			56	

Shortest paths: Exercises

Table 4-9. *Optimal Solution to Shortest Path Problem*

Points	0	3	7	9	10	12
Distance	0	24	17	14	28	6
Cumulative	0	24	41	55	83	89

```
>>> %Run shortest_path_2.py
```

```
Path: [0, 3, 7, 9, 10, 12]
```

```
Cost: [0, 24, 17, 14, 28, 6]
```

```
Cumulative cost: [0, 24, 41, 55, 83, 89]
```

```
Objective value: 89.0
```

Decision variables (X):

```
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Shortest paths: Alternate algorithms

- Tại sao không dùng thuật toán Dijkstra để tìm đường đi ngắn nhất mà dùng mô hình quy hoạch tuyến tính?
 - Mặc dù thuật toán Dijkstra triển khai nhanh hơn?
- Nhưng trong thực tế, mô hình tuyến tính không chỉ giải bài toán thuần túy “pure shortest path”
 - Có thể ràng buộc thêm nhiều điều kiện (bound to be multiple additional considerations)
 - Việc thêm ràng buộc vào mô hình tuyến tính đơn giản hơn nhiều so với việc thay đổi cài đặt thuật toán Dijkstra

Shortest paths: Variations

- Thay vì tìm cực tiểu tổng các khoảng cách (sum of distances), ta muốn tìm cực tiểu tích (product) các khoảng cách thì sao?
- Lưu ý
 - Không thể thực hiện phép nhân (product/multiply) các biến với linear solver.
- Lấy logarithm từng khoảng cách và tìm cực tiểu tổng các log
 - vì $\log(a * b) = \log a + \log b$

Shortest paths: Variations → Critical tasks

- Tìm “critical tasks” trong bài toán project management
 - Critical paths: đường đi tới hạn, nếu công việc nằm trên đường này bị trễ (delay) thì sẽ ảnh hưởng đến toàn bộ dự án.
- Dưới đây là kết quả xếp lịch (output) của bài toán project management với thời điểm bắt đầu (sớm nhất có thể) và kết thúc của mỗi task.
 - Tìm các critical tasks?

Table 2-8. One Optimal Solution to the Project Management Problem

Task	0	1	2	3	4	5	6	7	8	9	10	11
Start	0	3	0	3	9	0	9	16	26	21	24	23
End	3	9	3	5	11	7	16	21	28	28	28	28

Shortest paths: Variations → Critical tasks

Table 2-7. *Example of Project Management Tasks*

Task	Duration	Preceding Tasks
0	3	{ }
1	6	{ 0 }
2	3	{ }
3	2	{ 2 }
4	2	{ 1 2 3 }
5	7	{ }
6	7	{ 0 1 }
7	5	{ 6 }
8	2	{ 1 3 7 }
9	7	{ 1 7 }
10	4	{ 7 }
11	5	{ 0 }

Shortest paths: Variations → Critical tasks

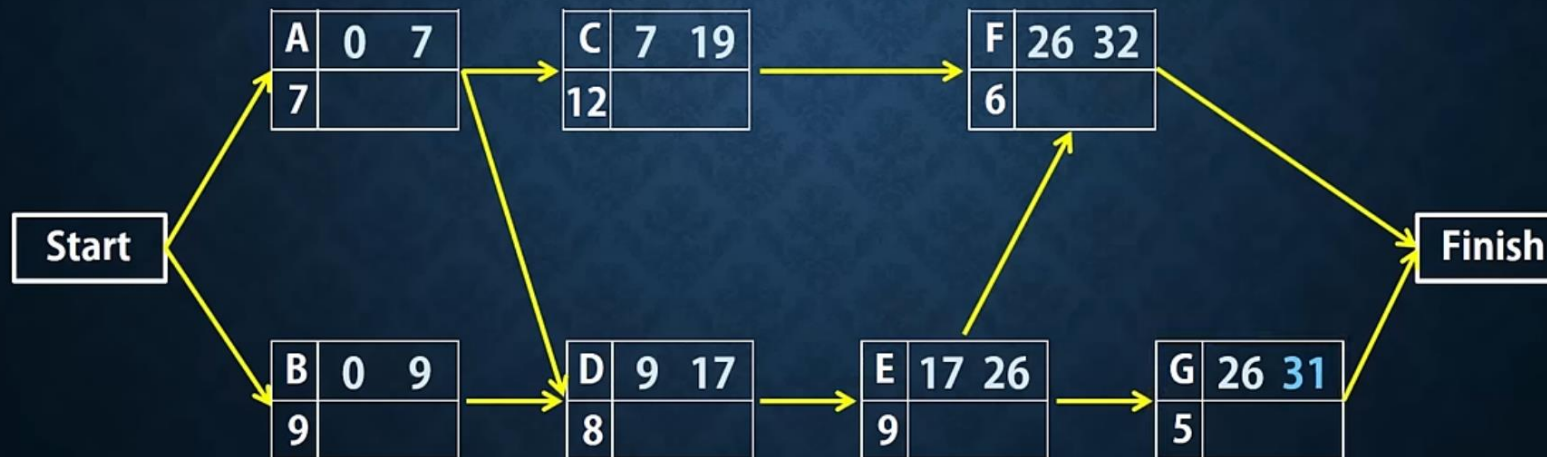
s		0	3	5	7	9	11	16	21	23	24	26	28
	M	N	1	2	3	4	5	6	7	8	9	10	11
0	N	N	-3	N	-7	N	N	N	N	N	N	N	N
3	1	N	N	-2	N	-6	N	N	N	N	N	N	N
5	2	N	N	N	N	N	N	N	N	N	N	N	N
7	3	N	N	N	N	N	N	N	N	N	N	N	N
9	4	N	N	N	N	N	-2	-7	N	N	N	N	N
11	5	N	N	N	N	N	N	N	N	N	N	N	N
16	6	N	N	N	N	N	N	N	-5	N	N	N	N
21	7	N	N	N	N	N	N	N	N	N	N	N	-7
23	8	N	N	N	N	N	N	N	N	N	N	N	-5
24	9	N	N	N	N	N	N	N	N	N	N	N	-4
26	10	N	N	N	N	N	N	N	N	N	N	N	-2
28	11	N	N	N	N	N	N	N	N	N	N	N	N

Shortest paths: Variations → Critical tasks



Activity	A	B	C	D	E	F	G
Immediate Predecessors	--	--	A	A, B	D	C, E	E
Expected Time (weeks)	7	9	12	8	9	6	5

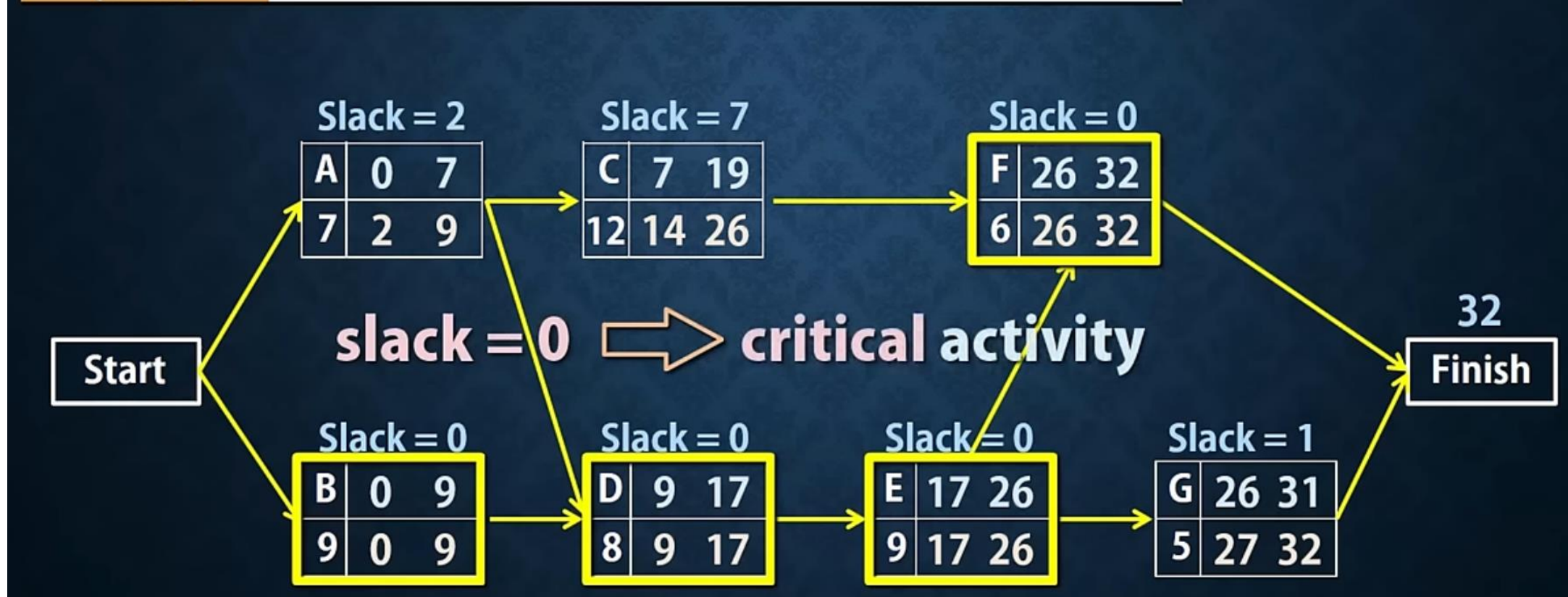
A	ES	EF
t	LS	LF



Shortest paths: Variations → Critical tasks

Activity	A	B	C	D	E	F	G
Immediate Predecessors	--	--	A	A, B	D	C, E	E
Expected Time (weeks)	7	9	12	8	9	6	5

A	ES	EF
t	LS	LF

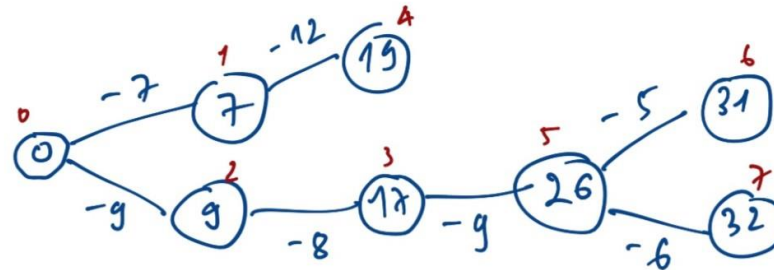


Shortest paths: Variations → Critical tasks

Critical path (using shortest path)

$i =$ 0 1 2 3 4 5 6
 $t[i]$ 0 0 7 9 17 26 26
 7 9 19 17 26 32 31

$S = \{0, 7, 9, 17, 19, 26, 31, 32\}$



B → D → E → F
 1 3 4 5
 start = 0 } → qua
 end = 32 } 9, 17, 26

	0	7	9	17	19	26	31	32
0		-7	-9					
7					-12			
9				-8				
17						-9		
19							-5	
26								-6
31								
32								

Shortest paths: Variations → Critical tasks

```
1  """
2  Sử dụng Network flow để xác định các công việc tới hạn (critical tasks)
3  trong bài toán Project Management.
4  (critical_tasks.py)
5  """
6  from shortest_path import solve_shortest_path
7
8
9  # D: bảng mô tả công việc (xem hình project_scheduling.png)
10 # t[i]: thời điểm bắt đầu của công việc thứ i
11 def critical_paths(D, t):
12     # s: tập hợp thời điểm bắt đầu và kết thúc các tất cả các công việc
13     # vì s là tập hợp nên không có phần tử trùng lặp, theo thứ tự tăng dần
14     # tập s sẽ trở thành các ndoe trong đồ thị
15     start_times = [t[i] for i in range(len(t))]
16     end_times = [t[i] + D[i][1] for i in range(len(t))]
17     s = set(start_times + end_times)
18     s = sorted(s) # sắp xếp tăng dần
```

Shortest paths: Variations → Critical tasks

```
20     n = len(s)
21     start = min(s)
22     end = max(s)
23     # times: dictionary gồm key: thời điểm bắt đầu của công việc,
24     # value: thứ tự/label của công việc trên đồ thị
25     times = {}
26     idx = 0 # index của node trên đồ thị
27     for e in s:
28         times[e] = idx
29         idx += 1
30
31     # Tạo ma trận khoảng cách giữa hai node thời điểm đầu và kết thúc của công việc
32     M = [[None for _ in range(n)] for _ in range(n)]
33     for i in range(len(t)):
34         # do tìm đường dài nhất nên đổi dấu -D[i][1]
35         M[times[t[i]]][times[t[i] + D[i][1]]] = -D[i][1]
```

Shortest paths: Variations → Critical tasks

```
37 status, obj_val, X, Path, cost, cumulative_cost \  
38 |     = solve_shortest_path(M, times[start], times[end])  
39 print('obj_val:', obj_val)  
40 print('Path:', Path)  
41 T = [i for i in range(len(t)) for time in Path if times[t[i] + D[i][1]] == time]  
42 ''' tương đương code sau  
43 T = []  
44 for i in range(len(t)):  
45 |     for time in Path:  
46 |         if times[t[i] + D[i][1]] == time:  
47 |             T.append(i)  
48 ...  
49 return status, T
```


Shortest paths: Variations → Critical tasks

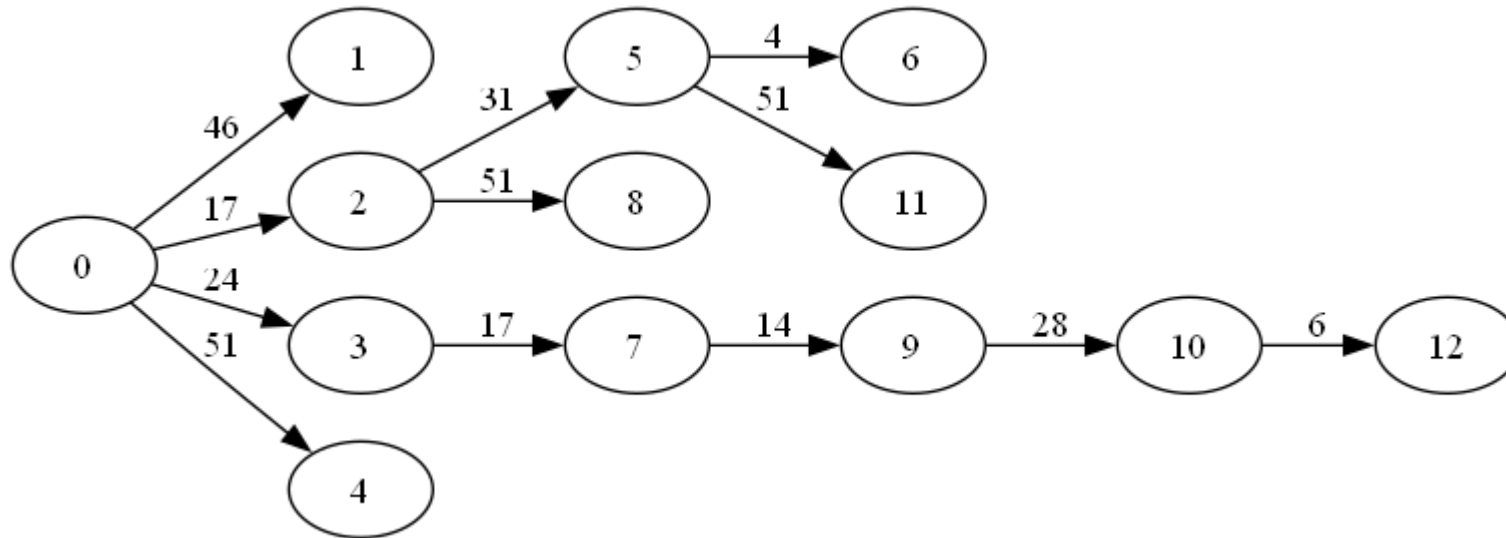
```
52 ∨ def main():
53     # D[i] = [task i, duration, {preceding tasks of task i}]
54 ∨     D = [[0, 7, {}],
55            [1, 9, {}],
56            [2, 12, {0}],
57            [3, 8, {0, 1}],
58            [4, 9, {3}],
59            [5, 6, {2, 4}],
60            [6, 5, {4}]]
61     t = [0, 0, 7, 9, 17, 26, 26]
62
63     status, T = critical_paths(D, t)
64 ∨     if status != 0:
65         print('Infeasible')
66 ∨     else:
67         print('Critical tasks (T):', T)
```

obj_val: -32.0
Path: [0, 2, 3, 5, 7]
Critical tasks (T): [1, 3, 4, 5]

Tương ứng các task
[B, D, E, F] trên hình

Shortest paths: Variations → Shortest paths tree

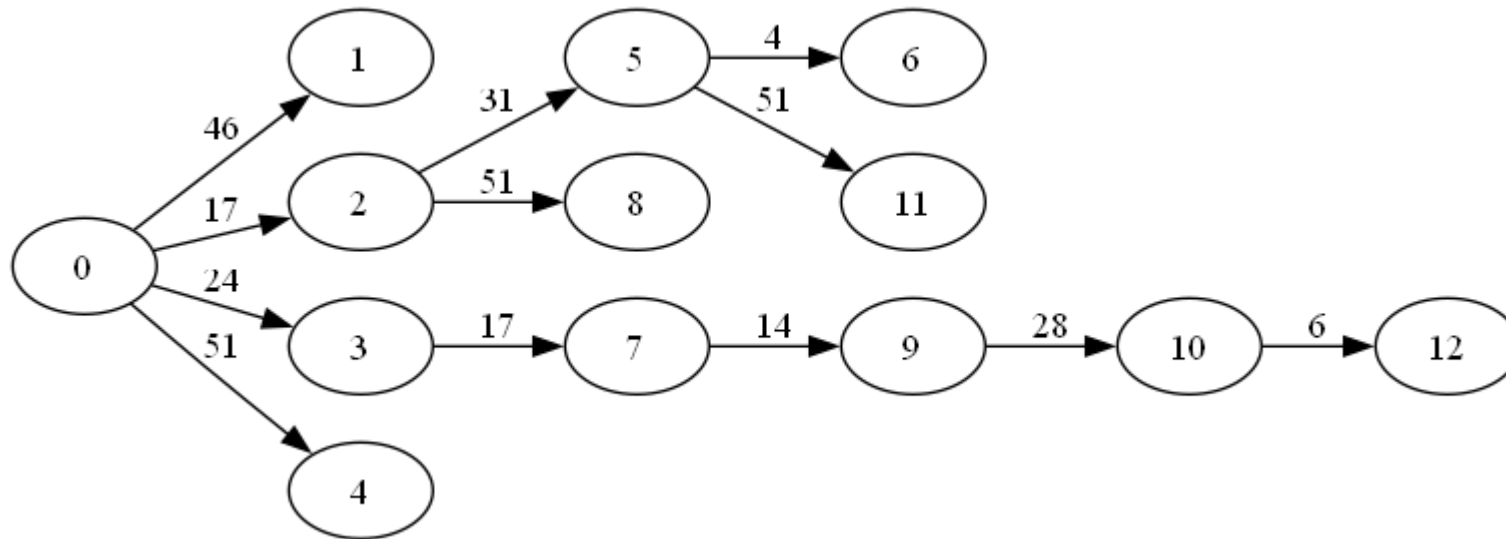
- Muốn tìm đường đi ngắn nhất từ một điểm bắt đầu (start node/source) đến tất cả các điểm còn lại (other node) trên mạng thì sao?
 - Chạy chương trình với từ điểm bắt đầu → đến từng điểm còn lại → có tối đa $(n - 1)$ shortest paths
- Tuy nhiên, có cách trình bày kết quả ngắn gọn hơn dưới dạng cây
 - Shortest path từ 0 → 8: qua node 2 với distance = $17 + 51 = 68$
 - Shortest path từ 0 → 12: qua các node [3, 7, 9, 10] với distance = 89



Shortest paths: Variations → Shortest paths tree

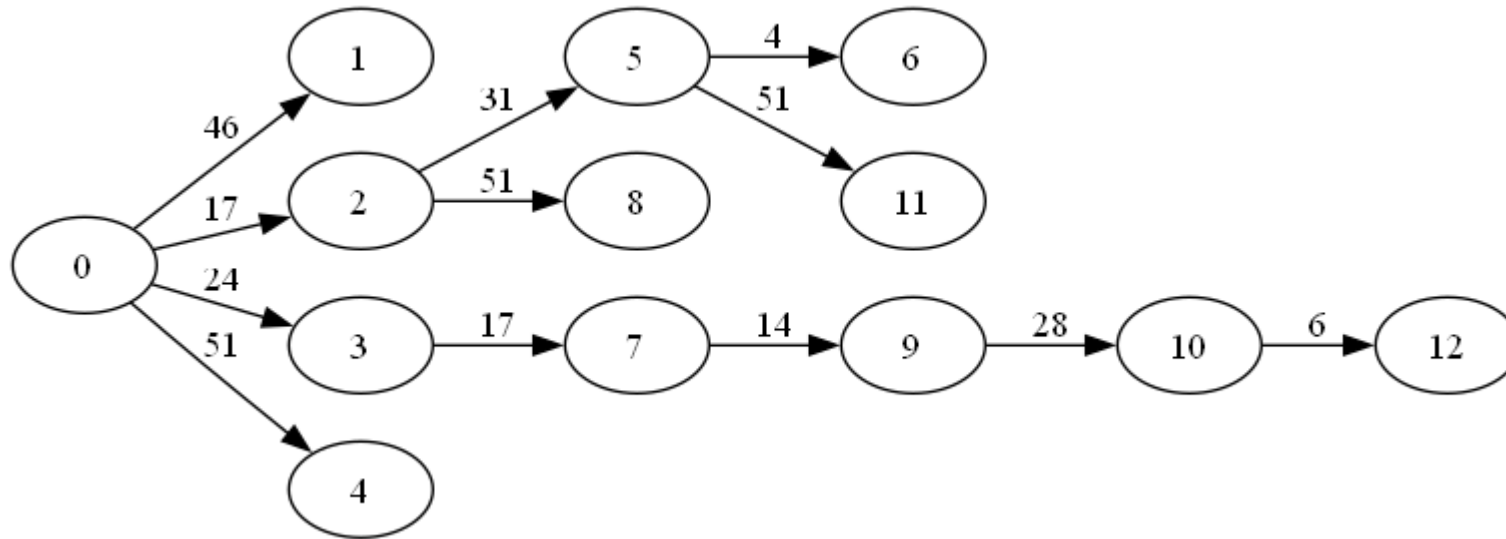
■ Ý tưởng minimize network flow như sau

- Ví dụ từ node 0 đến các node [2, 5, 6, 8, 11] gồm 5 shortest paths
 - $0 \rightarrow 2$
 - $0 \rightarrow 2 \rightarrow 5$
 - $0 \rightarrow 2 \rightarrow 5 \rightarrow 6$
 - $0 \rightarrow 2 \rightarrow 5 \rightarrow 11$
 - $0 \rightarrow 2 \rightarrow 8$



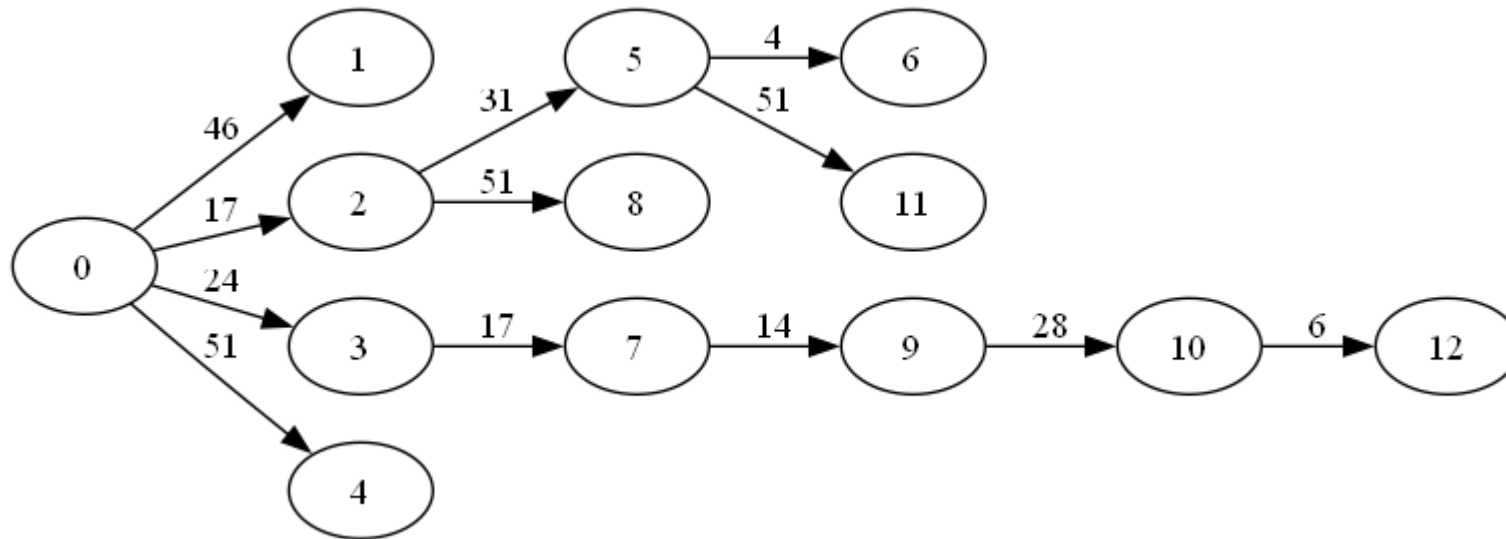
Shortest paths: Variations → Shortest paths tree

- Vậy tổng khoảng cách của 5 shortest path trên gồm 5 lần quãng đường từ $0 \rightarrow 2$, cộng cho 3 lần quãng đường từ $2 \rightarrow 5$, cộng 1 lần quãng đường từ $2 \rightarrow 8$, cộng 1 lần quãng đường từ $5 \rightarrow 6$, và cộng 1 lần quãng đường từ $5 \rightarrow 11$
- Nếu gọi $G_{i,j}$ là số quãng đường từ node i đến node j được sử dụng
 - $G_{0,2} = 5, G_{2,5} = 3, G_{2,8} = 1, G_{5,6} = 1, G_{5,11} = 1$
- Tổng các quãng đường (distances) hay chi phí (cost)
 - $G_{i,j} * D_{i,j}$



Shortest paths: Variations → Shortest paths tree

- Bài toán thành cực tiểu tổng distances/cost các shortest paths từ node 0 đến tất cả các node còn lại trên mạng
 - Network flow chỉ có 1 source node (hay supply node)
 - Các node còn lại là demand node



Shortest paths: Variations → Shortest paths tree

Constructing a model

■ Decision variables

- $G_{i,j}$: số lượng shortest paths từ node i đến các node nào đó có đi qua node j (tính luôn cả node j)
- Ví dụ: $G[2][5] = 3 \rightarrow$ có 3 shortest paths từ node 2 đến các node [5, 6, 11] có đi qua node 5

$$0 \leq G_{i,j} \leq n$$

■ Constraints

$$\sum_j G_{i,j} = n - 1 \quad i \in \text{source node}$$

$$\sum_j G_{j,i} = 0 \quad i \in \text{source node}$$

$$G_{j,i} - G_{i,j} = 1 \quad \forall i \neq \text{source node}$$

Shortest paths: Variations → Shortest paths tree

- Objective function

$$\min \sum_i \sum_j G_{i,j} \times D_{i,j}$$

Shortest paths: Variations → Shortest paths tree



```
1  """
2  Tìm cây đường đi ngắn nhất từ một đỉnh đến tất cả các đỉnh còn lại trong đồ thị
3  sử dụng Linear Network Models Optimization
4  (shortest_paths_tree.py)
5  """
6  import numpy as np
7  from ortools.linear_solver import pywraplp
8  from my_or_tools import *
9
10
11  # D: distance matrix
12  def solve_shortest_path_tree(D, start=None):
13      solver = pywraplp.Solver('Shortest Path Tree Problem',
14                              pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
15      n = len(D)
16      start = 0 if start is None else start
```

Shortest paths: Variations → Shortest paths tree



```
18 # Decision variables
19 # G[i][j]: số lượng shortest paths từ node i đến các node nào đó có đi qua node j
20 # (tính luôn cả node j). Ví dụ: G[2][5] = 3 ☞ có 3 shortest paths từ node 2 đến
21 # các node [5, 6, 11] có đi qua node 5
22 G = [[solver.NumVar(0, 0 if D[i][j] is None else n, '')
23       | for j in range(n)] for i in range(n)]
24
25 # Constraints
26 for i in range(n):
27     if i == start:
28         # start node/source: như là supply node, chỉ có n - 1 output flow
29         # tất nhiên không có input flow
30         solver.Add(sum(G[start][j] for j in range(n)) == n - 1)
31         solver.Add(sum(G[j][start] for j in range(n)) == 0) # source/supply node
32     else:
33         # số lượng shortest paths vào node i - ra khỏi node i bằng 1
34         # (do bỏ qua node i)
35         solver.Add(sum(G[j][i] for j in range(n)) -
36                     sum(G[i][j] for j in range(n)) == 1)
```

Shortest paths: Variations → Shortest paths tree



```
38     # Objective function
39     # Minimize tổng khoảng cách của tất cả các shortest paths
40     solver.Minimize(solver.Sum(G[i][j] * (0 if D[i][j] is None else D[i][j])
41     |         |         |         |         |         |         |         for i in range(n) for j in range(n)))
42     status = solver.Solve()
43
44     SP_Tree = [[i, j, D[i][j]] for i in range(n) for j in range(n)
45     |         |         |         if G[i][j].solution_value() > 0]
46
47     return status, ObjVal(solver), SolVal(G), SP_Tree
48
```


Shortest paths: Variations → Shortest paths tree



```
50 def main():
51     # Đọc ma trận khoảng cách từ file csv
52     D = np.array(read_int_matrix('data/shortest_path.csv'))
53
54     status, obj, G, SP_Tree = solve_shortest_path_tree(D)
55     if status == pywraplp.Solver.OPTIMAL:
56         print('Optimal objective value =', obj)
57         print('Shortest Paths Tree')
58         for node in SP_Tree:
59             print(node)
60
61         print('Decision variables')
62         for i in range(len(G)):
63             for j in range(len(G[i])):
64                 if G[i][j] > 0:
65                     print(f'G[{i}][{j}] = {G[i][j]}')
66     else:
67         print('The problem does not have an optimal solution.')
68
```

Shortest paths: Variations → Shortest paths tree



Optimal objective value = 673.0

Shortest Paths Tree

[0, 1, 46]
[0, 2, 17]
[0, 3, 24]
[0, 4, 51]
[2, 5, 31]
[2, 8, 51]
[3, 7, 17]
[5, 6, 4]
[5, 11, 51]
[7, 9, 14]
[9, 10, 28]
[10, 12, 6]

Decision variables

$G[0][1] = 1.0$
 $G[0][2] = 5.0$
 $G[0][3] = 5.0$
 $G[0][4] = 1.0$
 $G[2][5] = 3.0$
 $G[2][8] = 1.0$
 $G[3][7] = 4.0$
 $G[5][6] = 1.0$
 $G[5][11] = 1.0$
 $G[7][9] = 3.0$
 $G[9][10] = 2.0$
 $G[10][12] = 1.0$

